

НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Факультет Программной Инженерии и Компьютерной Техники

Системы компьютерной обработки изображений

Лабораторная работа № 4

«Формирование распределения случайных величин в заданной области
определения»

Выполнил
студент

Стрельбицкий Илья Павлович

Группа № Р3413

Преподаватель: Жданов Дмитрий Дмитриевич

г. Санкт-Петербург 2026

Цель

Изучить методы формирования заданных распределений случайных координат и направлений лучей в пространстве. Доказать корректность полученных распределений случайных величин.

Задание

1. Сформировать равномерное распределение случайных точек ($P_1, P_2, \dots$) внутри треугольника. Треугольник задан тремя координатами в пространстве (V_1, V_2, V_3). Число точек используемых в выборке 100000, число точек, сформированных внутри треугольника также 100000.

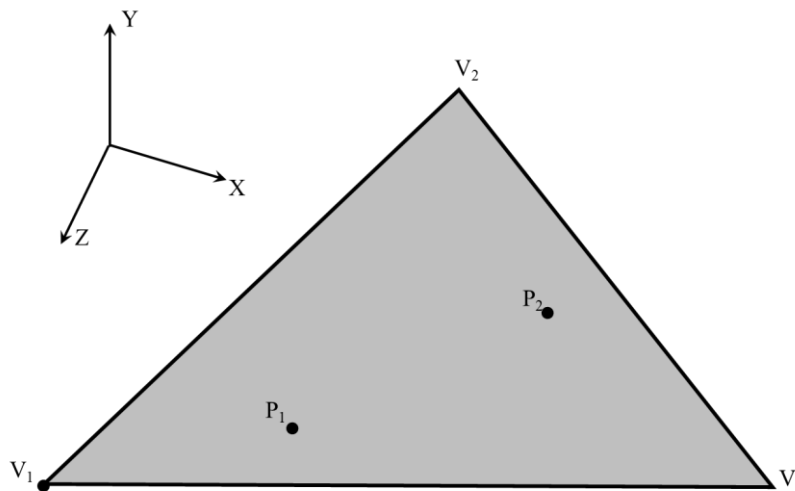


Рисунок 1.1 Задание 1

Доказать, что полученное распределение равномерное и все точки лежат внутри треугольника.

Решение:

Алгоритм генерации точек.

- С помощью функции `rand` генерируем два набора координат u и v в диапазоне $[0, 1]$. Получаем квадрат из точек.
- Точки, выходящие за диагональ квадрата «отражаем» вниз: $u=1-u$, $v=1-v$. Получаем треугольник из точек.
- Деформируем треугольник в координаты P_0, P_1, P_2 .

Полученная 2д проекция треугольника – рисунок 1.2.

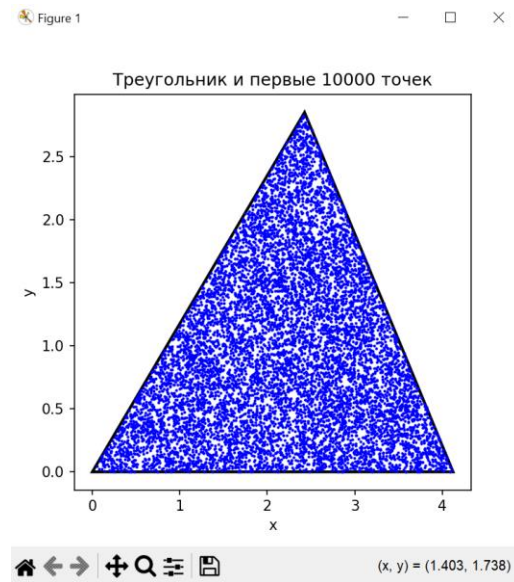


Рисунок 1.2. Визуализация треугольника.

Для того чтобы проверить равномерность распределения точек, построим тепловую карту распределения.

Для этого реализуем следующий алгоритм:

- Разбиваем треугольник на регулярную сетку, например 100 на 100 ячеек, и посчитаем, сколько точек попало в каждую ячейку.
- Строим тепловую карту плотности точек – рисунок 1.3.

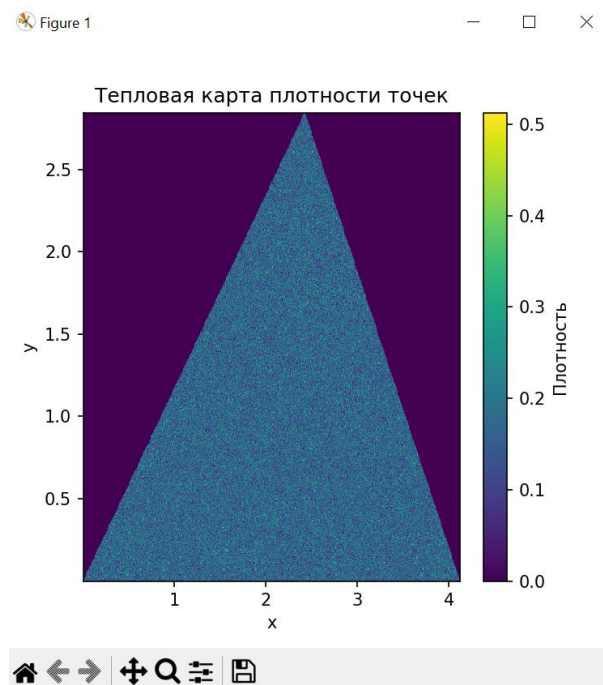


Рисунок 1.3. Карта распределения плотности точек

По рисунку видно, что зон с аномальным количеством точек нет, следовательно распределение равномерное.

Исходный код на python:

```
import numpy as np
import matplotlib.pyplot as plt

def generate_sample_triangle(v1: np.ndarray, v2: np.ndarray, v3: np.ndarray,
n: int) -> np.ndarray:
    """
    Функция генерации равномерных точек внутри треугольника
    Метод: u, v — независимые равномерные, но если u+v>1 — отражаем
    """
    u = np.random.rand(n)
    v = np.random.rand(n)
    # отражение для равномерности
    mask = u + v > 1
    u[mask] = 1 - u[mask]
    v[mask] = 1 - v[mask]
    # генерация точек
    return v1 + u[:,None] * (v2 - v1) + v[:,None] * (v3 - v1)

def visualization_dots(triangle_projection, projection_x, projection_y, n:
int = 10000) -> None:
    """Визуализация треугольника с отмеченными n точек"""
    plt.figure(figsize=(5, 5))
    plt.fill(
        triangle_projection[:, 0],
        triangle_projection[:, 1],
        edgecolor='black',
        fill=False,
        linewidth=2
    )
    plt.scatter(projection_x[:n], projection_y[:n], s=1, color='blue')
    plt.title(f"Треугольник и первые {n} точек")
    plt.xlabel("x")
    plt.ylabel("y")
    plt.show()

def visualization_of_distribution_heat_map(projection_x, projection_y) ->
None:
    """Визуализация тепловой карты распределения"""
    plt.figure(figsize=(5, 5))
    plt.hist2d(projection_x, projection_y, bins=500, density=True)
    plt.title("Тепловая карта плотности точек")
    plt.xlabel("x")
    plt.ylabel("y")
    plt.colorbar(label="Плотность")
    plt.show()

def main() -> None:
```

```

# число точек
n = 1000000
# координаты треугольника
v1 = np.array([1, 2, 0])
v2 = np.array([3, 5, 2])
v3 = np.array([0, 4, 3])
points = generate_sample_triangle(v1, v2, v3, n)
# Чтобы визуализировать распределение, надо перевести точки в 2D (систему
координат треугольника)
e1 = v2 - v1
e2 = v3 - v1
# ортонормируем базис
e1n = e1 / np.linalg.norm(e1)
e2p = e2 - np.dot(e2, e1n) * e1n
e2n = e2p / np.linalg.norm(e2p)
# проекция на 2d
projection_x = np.dot(points - v1, e1n)
projection_y = np.dot(points - v1, e2n)
triangle_projection = np.array([
    np.dot(v1 - v1, e1n), np.dot(v1 - v1, e2n)],
    np.dot(v2 - v1, e1n), np.dot(v2 - v1, e2n)],
    np.dot(v3 - v1, e1n), np.dot(v3 - v1, e2n)],
])
# визуализация
visualization_dots(triangle_projection, projection_x, projection_y)
visualization_of_distribution_heat_map(projection_x, projection_y)

if __name__ == "__main__":
    main()

```

2. Сформировать равномерное распределение случайных точек ($P_1, P_2, \dots$) внутри круга. Круг задан ориентацией нормали N , радиусом круга R_c , и его центром C . Число точек используемых в выборке 100000, число точек, сформированных внутри круга также 100000.

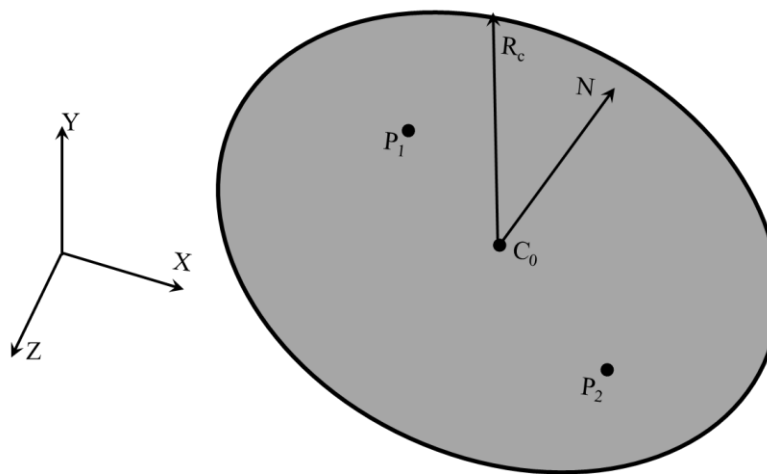


Рисунок 2.1. Задание 2.

Доказать, что полученное распределение равномерное и все точки лежат внутри круга.

Решение:

Генерируем равномерно распределенные точки внутри диска, используя следующие формулы: $r = R * \sqrt{u}$, $\theta = 2 * \pi * u$.

Где R – максимальный радиус, u – случайное число $[0, 1]$.

Затем переводим их в локальные координаты через синус и косинус угла и гипотенузу. Окружность со сгенерированными точками видны на рисунке 2.2.

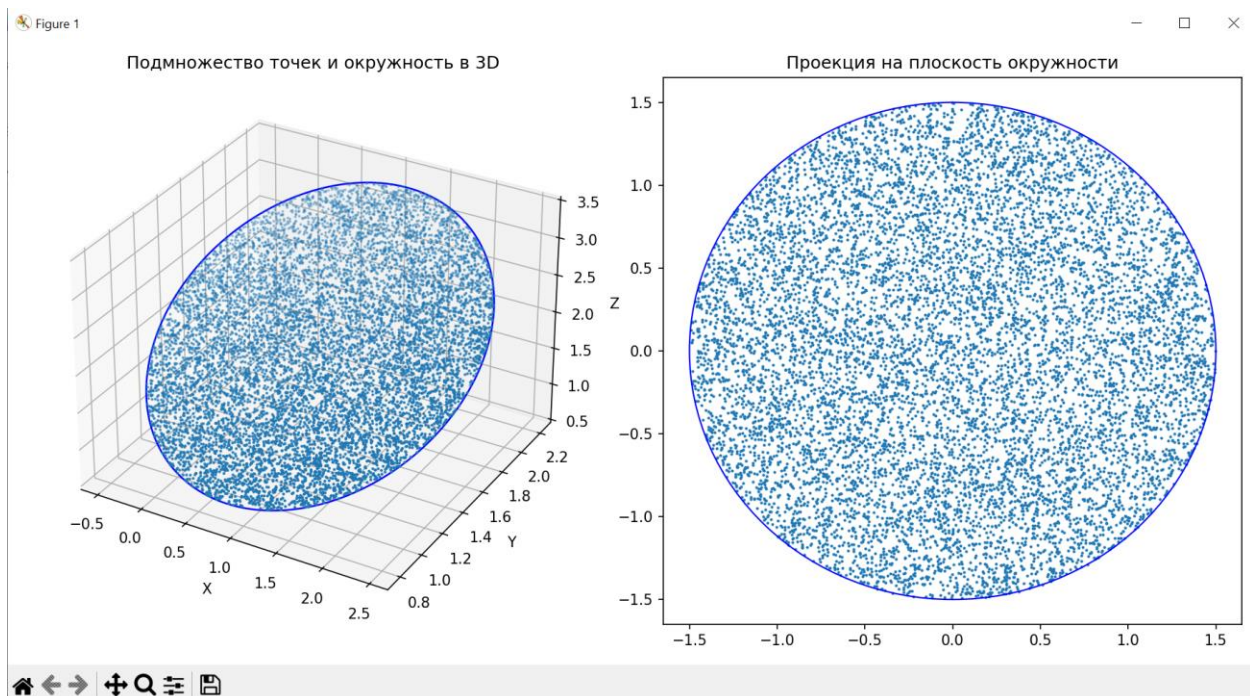


Рисунок 2.2. Окружность со сгенерированными точками (первые 10 000 точек)

Для доказательства равномерности распределения точек построим график радиального распределения – рисунок 2.3.

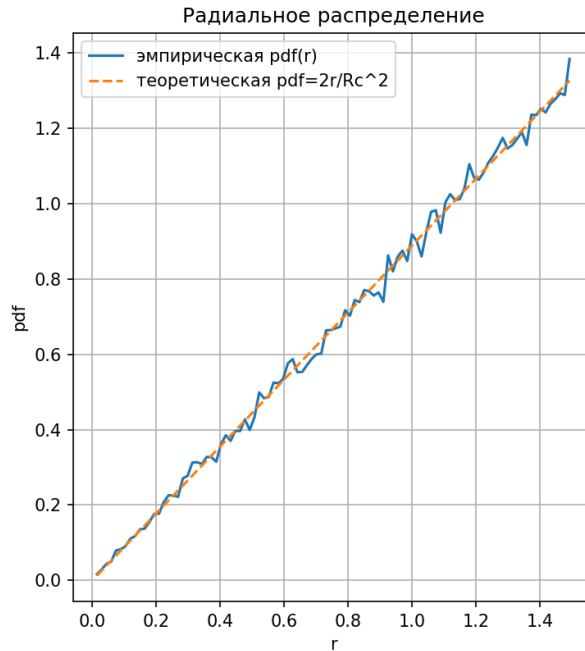


Рисунок 2.3. График радиального распределения

Также в программе проведем дополнительные расчеты, результаты которых видны на рисунке 2.4.

```
D:\university\spi\fourth\.venv\Scripts\python.exe D:\university\spi\fourth\circle.py
Количество точек: 100000
Результат проверки статистической равномерности:
min:40, max: 4000, mean: 2000.0
статистика плотностей:
средняя плотность = 14223.758812490874
стандартное отклонение = 489.2006714169297
Контрольные проверки:
Максимальный радиус меньше или равен Rc: True
Коэффициент вариации плотности: 0.03439320631529048
CV < 0.1 обычно приемлемо для равномерности
```

Рисунок 2.4. Вывод программы

Исходный код на Python:

```
import numpy as np
import matplotlib.pyplot as plt

def normalize(v):
    n = np.linalg.norm(v)
    if n < 1e-12:
        raise ValueError("Вектор нулевой длины")
```

```

return v / n

def generate_points(C, Rc, number_points, u_axis, v_axis):
    """Генерируем равномерно распределенные точки внутри диска"""
    u = np.random.rand(number_points)
    theta = 2 * np.pi * np.random.rand(number_points)
    r = Rc * np.sqrt(u)
    x_local = r * np.cos(theta)
    y_local = r * np.sin(theta)
    return C[None, :] + np.outer(x_local, u_axis) + np.outer(y_local, v_axis)

def check_uniformity(Rc, radial, counts, density) -> None:
    """Проверка статистической равномерности"""
    print("Результат проверки статистической равномерности:")
    print(f"min:{counts.min()}, max: {counts.max()}, mean: {counts.mean()}")
    print("статистика плотностей:")
    print(f"средняя плотность = {density.mean()}")
    print(f"стандартное отклонение = {density.std()}")
    # Вывод результатов
    print("Контрольные проверки:")
    print(f"Максимальный радиус меньше или равен Rc: {radial.max() <= Rc + 1e-12}")
    cv = density.std() / density.mean()
    print(f"Коэффициент вариации плотности: {cv}")
    print("CV < 0.1 обычно приемлемо для равномерности" if cv < 0.1 else "CV слишком велик")

def draw_first_figure(
    C,
    Rc,
    number_points,
    points,
    u_axis,
    v_axis,
    x_projection,
    y_projection,
    number_points_for_draw=10000
) -> None:
    """Визуализация генерации точек в плоскости круга"""
    subset_idx = np.random.choice(number_points, size=number_points_for_draw,
replace=False)
    pts3 = points[subset_idx]
    # Подмножество точек и окружность в 3D
    first_figure = plt.figure(figsize=(12, 6))
    graph_first = first_figure.add_subplot(1, 2, 1, projection='3d')
    graph_first.scatter(pts3[:, 0], pts3[:, 1], pts3[:, 2], s=1)
    angles = np.linspace(0, 2 * np.pi, 200)
    rim = C[None, :] + np.outer(Rc * np.cos(angles), u_axis) + np.outer(Rc *
np.sin(angles), v_axis)
    graph_first.plot(rim[:, 0], rim[:, 1], rim[:, 2], 'b', linewidth=1.2)
    graph_first.set_title("Подмножество точек и окружность в 3D")
    graph_first.set_xlabel('X')
    graph_first.set_ylabel('Y')
    graph_first.set_zlabel('Z')
    # Проекция на плоскость окружности
    graph_second = first_figure.add_subplot(1, 2, 2)
    graph_second.scatter(x_projection[subset_idx], y_projection[subset_idx],

```



```

s=1)
    circle = plt.Circle((0, 0), Rc, edgecolor='b', facecolor='none')
    graph_second.add_patch(circle)
    graph_second.set_xlim(-Rc * 1.1, Rc * 1.1)
    graph_second.set_ylim(-Rc * 1.1, Rc * 1.1)
    graph_second.set_title("Проекция на плоскость окружности")
    plt.tight_layout()

def draw_second_figure(Rc, radial) -> None:
    """Распределение плотностей генерируемых точек"""
    second_figure = plt.figure(figsize=(12, 6))
    # pdf(r) - функция плотности вероятности
    graph_third = second_figure.add_subplot(1, 2, 1)
    hist_counts, hist_bins = np.histogram(radial, bins=100, density=True)
    bin_centers_second = 0.5 * (hist_bins[:-1] + hist_bins[1:])
    theo = 2 * bin_centers_second / (Rc ** 2)
    graph_third.plot(bin_centers_second, hist_counts, label='эмпирическая
pdf(r)')
    graph_third.plot(bin_centers_second, theo, '--', label='теоретическая
pdf=2r/Rc^2')
    graph_third.set_xlabel('r')
    graph_third.set_ylabel('pdf')
    graph_third.set_title('Радиальное распределение')
    graph_third.legend()
    graph_third.grid(True)

def main() -> None:
    # Исходные параметры
    C = np.array([1.0, 1.5, 2.0]) # центр окружности в 3D
    vector_n = np.array([0.2, -1.0, 0.5]) # вектор нормали (ориентация)
    Rc = 1.5 # радиус
    number_points = 100000 # количество точек
    # Рассчитываем ортогональный базис (u_axis, v_axis) для плоскости круга
    Nn = normalize(vector_n)
    if abs(Nn[0]) < 0.9:
        tmp = np.array([1.0, 0.0, 0.0])
    else:
        tmp = np.array([0.0, 1.0, 0.0])
    u_axis = normalize(np.cross(Nn, tmp))
    v_axis = normalize(np.cross(Nn, u_axis))
    points = generate_points(C, Rc, number_points, u_axis, v_axis)
    # Проверка расстояний
    rel = points - C[None, :]
    x_projection = np.dot(rel, u_axis)
    y_projection = np.dot(rel, v_axis)
    radial = np.sqrt(x_projection**2 + y_projection**2)
    n_bins = 50
    bins = np.linspace(0, Rc, n_bins + 1)
    counts, _ = np.histogram(radial, bins=bins)
    areas = np.pi * (bins[1:] ** 2 - bins[:-1] ** 2)
    density = counts / areas
    print("Количество точек:", number_points)
    check_uniformity(Rc, radial, counts, density)
    draw_first_figure(C, Rc, number_points, points, u_axis, v_axis,
x_projection, y_projection)
    draw_second_figure(Rc, radial)
    plt.show()

```

```
if __name__ == '__main__':
    main()
```

3. Сформировать равномерное распределение случайных направлений в пространстве (точек на единичной сфере $P_1, P_2, \dots$). Число точек используемых в выборке 100000, число полученных направлений также 100000.

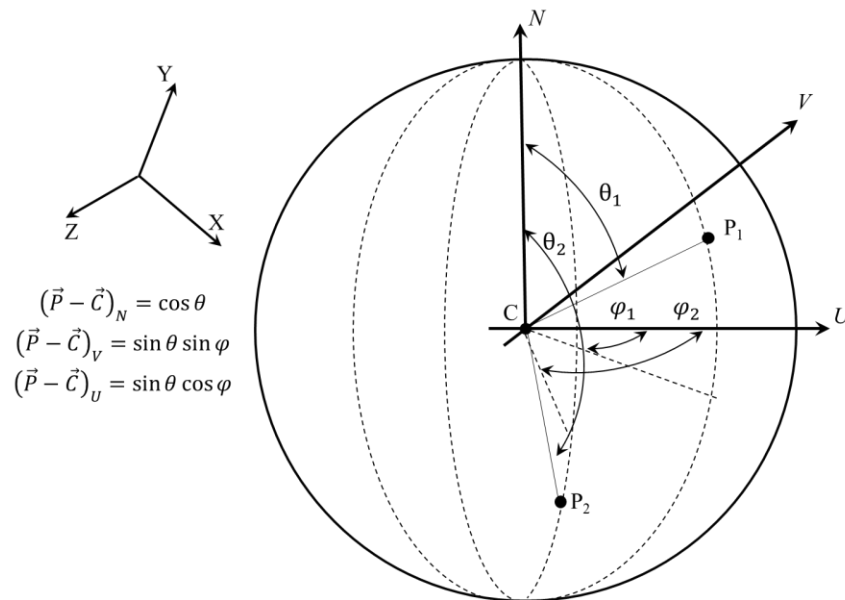


Рисунок 3.1. Задание 3.

Доказать, что полученное распределение равномерное и для всех выборок были сформированы направления.

Получаем следующую визуализацию сферы и точек – рисунок 3.2.

Figure 1

Визуализация сферы и точек (10000 точек)

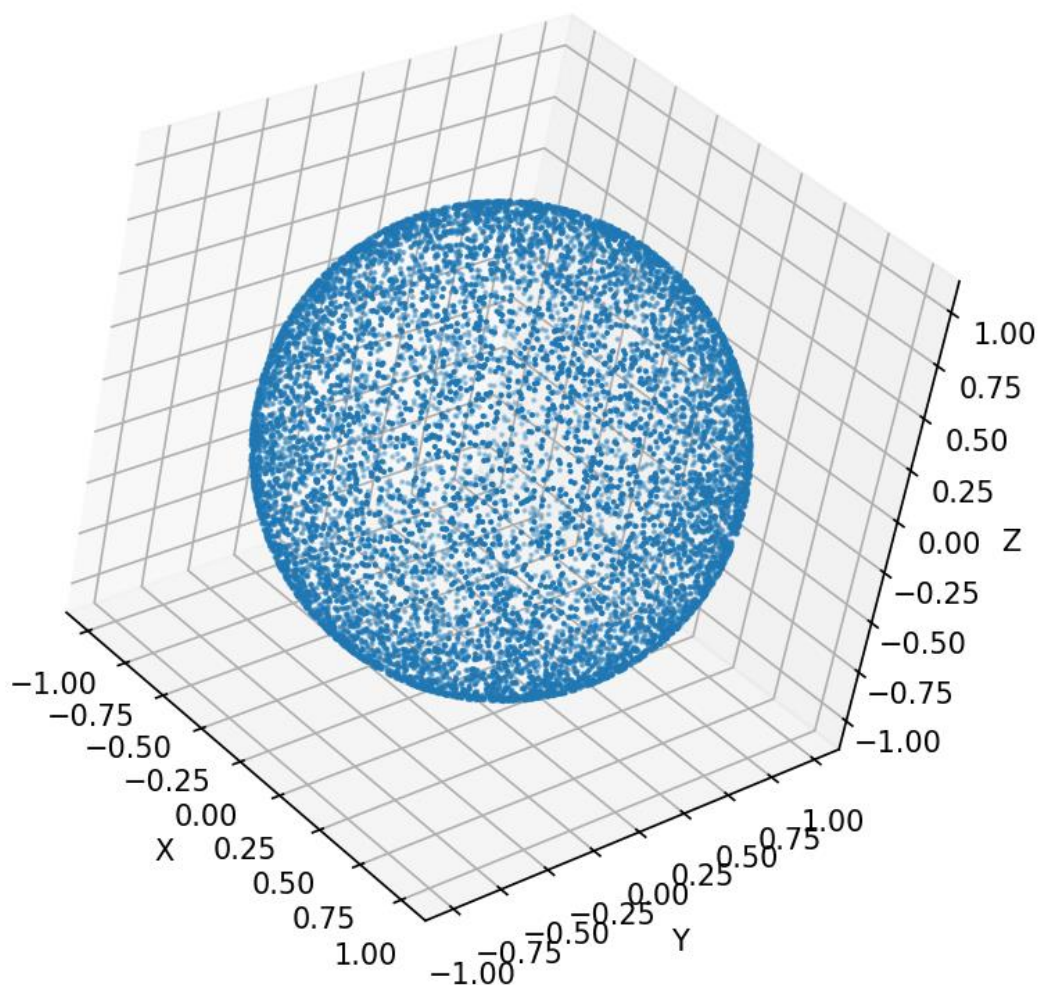


Рисунок 3.2. Визуализация сферы и точек.

Для проверки равномерности, рассчитаем среднее значение отклонения от радиуса – рисунок 3.3.

```
D:\university\spi\fourth\.venv\Scripts\python.exe D:\university\spi\fourth\sphere.py
Среднее отклонение радиуса от 1: 2.779554364451542e-17
```

Рисунок 3.3. Результат программы.

Также построим гистограмму углов отклонения, в идеальном случае она должна быть в виде ровной линии, в нашем случае она достаточно ровная, следовательно полученное распределение равномерное – рисунок 3.4.

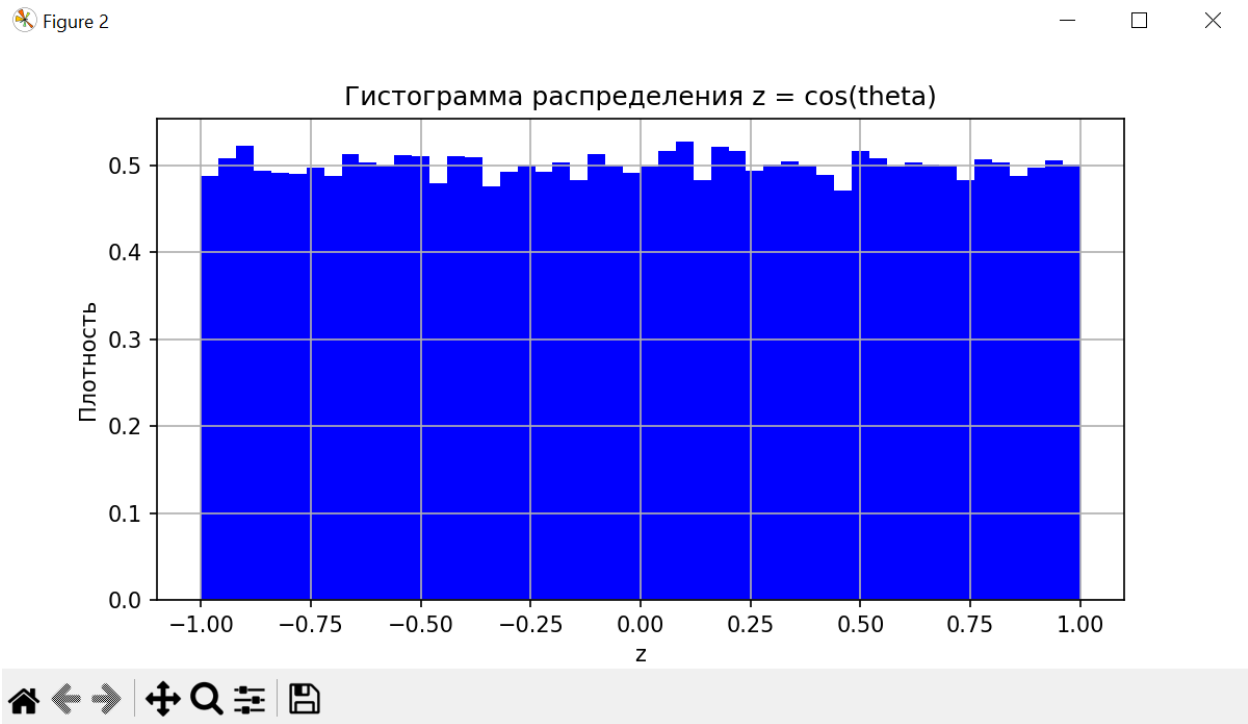


Рисунок 3.4. Гистограмма распределения углов отклонения.

Исходный код на Python:

```
import numpy as np
import matplotlib.pyplot as plt

def check_points(x, y, z) -> None:
    """Проверка того что все точки на сфере и расчет среднего отклонения от
    радиуса"""
    r = np.sqrt(x * x + y * y + z * z)
    print(f"Среднее отклонение радиуса от 1: {np.mean(np.abs(r - 1))}")

def draw_sphere(points, n: int = 10000) -> None:
    """Визуализация сферы и точек"""
    fig = plt.figure(figsize=(6, 6))
    ax = fig.add_subplot(111, projection='3d')
    sample = points[:n]
    ax.scatter(sample[:, 0], sample[:, 1], sample[:, 2], s=1)
    ax.set_title(f"Визуализация сферы и точек ({n} точек)")
    ax.set_box_aspect([1, 1, 1])
    ax.set_xlabel("X")
    ax.set_ylabel("Y")
    ax.set_zlabel("Z")
```

```

def draw_histogram(z) -> None:
    """
    Проверка равномерности распределения
    В идеальном случае гистограмма Z должна быть ровной линией
    """
    plt.figure(figsize=(8, 4))
    plt.hist(z, bins=50, density=True, color='b')
    plt.title("Гистограмма распределения z = cos(theta)")
    plt.xlabel("z")
    plt.ylabel("Плотность")
    plt.grid(True)

def main() -> None:
    n = 100000
    # cos(theta) равномерно в [-1, 1]
    u = np.random.uniform(-1, 1, n)
    # phi равномерно в [0, 2π]
    phi = np.random.uniform(0, 2*np.pi, n)
    theta = np.arccos(u)
    # декартовы координаты (точки на сфере)
    x = np.sin(theta) * np.cos(phi)
    y = np.sin(theta) * np.sin(phi)
    z = np.cos(theta)
    # shape points = (100000, 3)
    points = np.vstack((x, y, z)).T
    check_points(x, y, z)
    draw_sphere(points)
    draw_histogram(z)
    plt.show()

if __name__ == '__main__':
    main()

```

4. Сформировать распределение случайных направлений с косинусным распределением плотности вероятности относительно вектора N (единичных векторов, ориентированных по направлению $P_1 - C, P_2 - C, \dots$, плотность вероятности пропорциональна длине вектора $P - C$). Распределение симметрично относительно вектора N . Число точек используемых в выборке 100000, число полученных направлений также 100000.

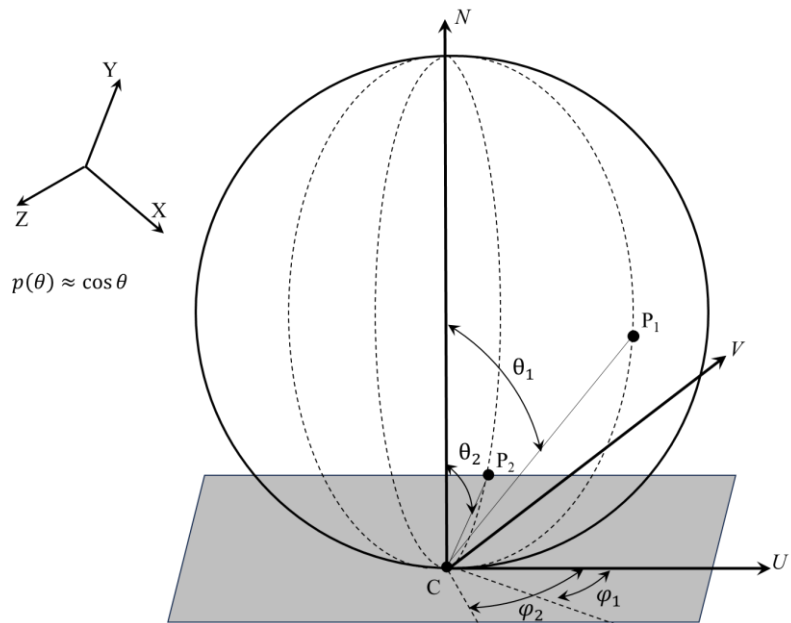


Рисунок 4.1. Задание 4.

Доказать, что полученное распределение косинусное и для всех выборок были сформированы направления.

Решение:

Для генерации случайных направлений с косинусным распределением (закон Ламберта) относительно вектора нормали N используется следующий подход:

Параметризация направлений.

Направление задается углами в сферической системе координат, где:

- α – угол между направлением и вектором N
- φ – азимутальный угол

Для косинусного распределения плотность вероятности пропорциональна $\cos(\alpha)$: $p(\alpha, \varphi) d\Omega \propto \cos(\alpha) \sin(\alpha) d\alpha d\varphi$, где $d\Omega = \sin(\alpha) d\alpha d\varphi$ – элемент телесного угла.

Для генерации равномерного распределения по полусфере с весом $\cos(\alpha)$:

Азимутальный угол равномерен на $[0, 2\pi]$

$\cos(\alpha)$ генерируется с распределением $p(\cos(\alpha)) = 2 * \cos(\alpha)$

Точки генерируются так, что длина вектора P-C пропорциональна $\cos(\alpha)$: $|P - C| = 2R * \cos(\alpha)$, где R – радиус сферы, C – исходная точка (южный полюс сферы), P – точка на сфере.

Для доказательства того, что полученное распределение косинусное построим гистограмму, где эмпирическое распределение $\cos(\alpha)$ будет представлено столбцами, а теоретическая кривая $f(c) = 2c$ синей линией. Визуализацию гистограммы, вместе с визуализацией точек сферы можно увидеть на рисунке 4.2.

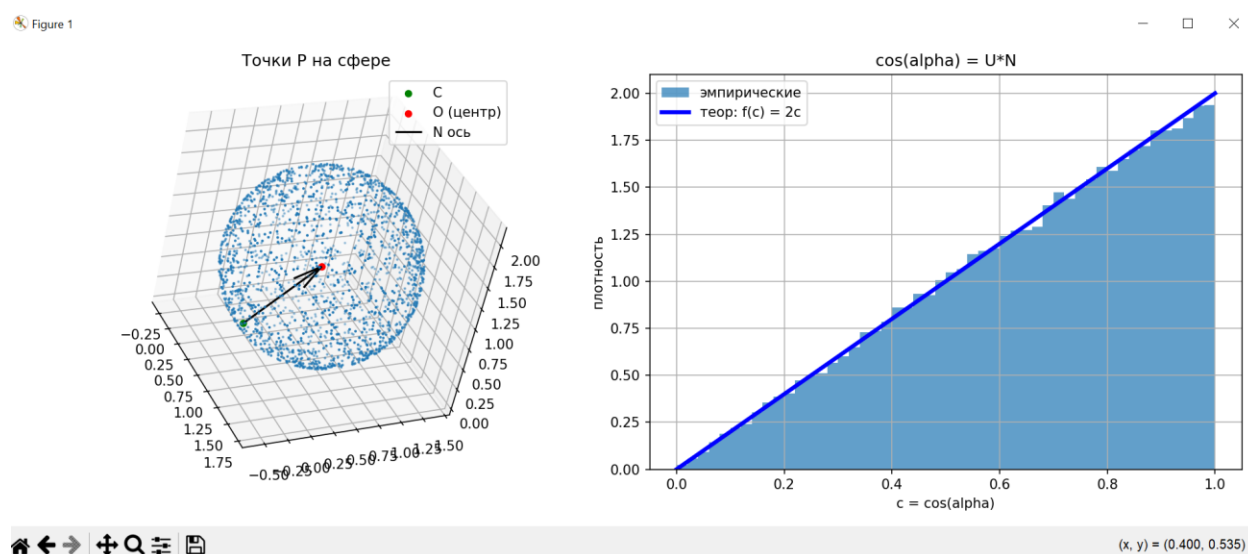


Рисунок 4.2. Визуализация точек сферы и гистограмма.

Из полученной гистограммы, видно, что совпадение хорошее, что подтверждает косинусное распределение.

Для проверки того, что все точки P лежат на сфере и то, что $\cos(\alpha)$ имеет распределение $p(c) = 2c$ на $[0, 1]$ сделаем дополнительные расчеты. Среднее эмпирическое значение $\cos(\alpha)$, должно быть максимально близким к теоретическому $2/3$. Результаты расчетов можно увидеть на рисунке 4.3.

```
D:\university\spi\fourth\.venv\Scripts\python.exe D:\university\spi\fourth\cosine.py
Расстояния до центра: 1.000000 (min), 1.000000 (max)
Все точки на сфере: True
cos(a): 0.004754 (min), 0.999989 (max)
E[cos(a)] = 0.665485 (теоретическое: 0.666667)
```

Рисунок 4.3. Результат программы.

Исходный код на Python:

```
import numpy as np
import matplotlib.pyplot as plt

def normalize(v):
    v = np.asarray(v, dtype=float)
    n = np.linalg.norm(v)
    if n < 1e-12:
        raise ValueError("Вектор нулевой длины")
    return v / n

def check(R, O, cos_alpha, P) -> None:
    # Проверка, что P действительно на сфере радиуса R вокруг O
    distances_to_center = np.linalg.norm(P - O, axis=1)
    print(f"Расстояния до центра: {distances_to_center.min():.6f} (min), {distances_to_center.max():.6f} (max)")
    print(f"Все точки на сфере: {np.allclose(distances_to_center, R, atol=1e-10)}")
    # cos(α) должен иметь распределение p(c) = 2c на [0, 1]
    print(f"cos(α): {cos_alpha.min():.6f} (min), {cos_alpha.max():.6f} (max)")
    print(f"E[cos(α)] = {cos_alpha.mean():.6f} (теоретическое: {2 / 3:.6f})")

def draw_first_figure(C, R, Nn, O, cos_alpha, P, n,
number_points_for_draw=2000) -> None:
    subset = np.random.choice(n, size=number_points_for_draw, replace=False)
    figure = plt.figure(figsize=(13, 5))
    # точки P на сфере
    graph_first = figure.add_subplot(1, 2, 1, projection="3d")
    graph_first.scatter(P[subset, 0], P[subset, 1], P[subset, 2], s=1)
    graph_first.scatter(C[0], C[1], C[2], s=20, color="g", label="C")
    graph_first.scatter(O[0], O[1], O[2], s=20, color="r", label="O (центр)")
    graph_first.quiver(C[0], C[1], C[2], Nn[0], Nn[1], Nn[2], length=R,
color="black", label="N ось")
    graph_first.set_title("Точки P на сфере")
    graph_first.set_box_aspect([1, 1, 1])
    graph_first.legend()
    # Гистограмма cos(alpha) и теоретическая плотность 2c
    graph_second = figure.add_subplot(1, 2, 2)
    bins = 50
    graph_second.hist(cos_alpha, bins=bins, range=(0, 1), density=True,
alpha=0.7, label="эмпирические")
    c_grid = np.linspace(0, 1, 500)
    graph_second.plot(c_grid, 2 * c_grid, linewidth=3, color="b",
label="теор: f(c) = 2c")
    graph_second.set_title("cos(alpha) = U*N")
    graph_second.set_xlabel("c = cos(alpha)")
    graph_second.set_ylabel("плотность")
    graph_second.grid(True)
    graph_second.legend()
    plt.tight_layout()
    plt.show()
```



```

def main() -> None:
    n = 100000
    C = np.array([0.5, -0.2, 0.3], dtype=float)
    vector_n = np.array([0.2, 0.6, 0.76], dtype=float) # ось N
    R = 1.0
    Nn = normalize(vector_n)
    # ортонормальный базис вокруг N
    # строим два вектора T, B такие, что {T, B, Nn} — ортонормальный базис
    if abs(Nn[2]) < 0.9:
        a = np.array([0.0, 0.0, 1.0])
    else:
        a = np.array([1.0, 0.0, 0.0])
    T = normalize(np.cross(a, Nn))
    B = np.cross(Nn, T) # уже нормированный из-за ортонормальности
    #  $p(\omega) = (\cos \alpha) / \pi$  на полусфере  $u \cdot N \geq 0$ 
    u1 = np.random.rand(n)
    u2 = np.random.rand(n)
    r = np.sqrt(u1)
    phi = 2 * np.pi * u2
    x = r * np.cos(phi)
    y = r * np.sin(phi)
    z = np.sqrt(1.0 - u1)
    # локальный вектор (x, y, z) -> мировой u = x*T + y*B + z*Nn
    U = (
        x[:, None] * T[None, :] +
        y[:, None] * B[None, :] +
        z[:, None] * Nn[None, :]
    )
    # дополнительно нормируем
    U = U / np.linalg.norm(U, axis=1)[:, None]
    # строим точки P на сфере, чтобы  $|P-C| \propto \cos(\alpha)$ 
    # центр сферы O = C + R * Nn (C — "южный полюс" сферы)
    O = C + R * Nn
    cos_alpha = U @ Nn # должно быть в [0,1]
    t = 2.0 * R * cos_alpha # это |P - C| (длина хорды), t ∈ [0, 2R]
    P = C + t[:, None] * U # пересечение луча со сферой (кроме t=0)
    # Проверка и графики
    check(R, O, cos_alpha, P)
    draw_first_figure(C, R, Nn, O, cos_alpha, P, n)

if __name__ == "__main__":
    main()

```